

Rafael Chetata
Big Data
Project 3
December 21, 2025

For the classification of cancer diagnoses as benign or malignant using PySpark ML algorithms, I chose Random Forest and the Multilayer Perceptron. In order to process the data, I split it into a test and training datasets, with a 80/20 split respectively. I then used a StringIndexer for the diagnosis column to convert the 'M' and 'B' values into numbers, which are value inputs. Then the features were transformed into a single vector through use of the VectorAssembler. For both models, I used a pipeline, which contained the label_indexer, assembler, and chosen model. The pipeline was fitted onto the data. Then, the model transformed the test data, and the predictions were evaluated.

I chose the Random Forest classifier initially because of its ability to capture nonlinear relationships with minimal preprocessing. Additionally, it is easy to understand, which makes it useful for communicating to others. This is further emphasized by the ability to track the splits (as seen in my code). This transparency can reveal trends in the qualities of malignant tumors and is easier to trust than black-box algorithms.

The Multilayer Perceptron classifier is a feedforward neural network capable of learning complex nonlinear decision boundaries. It was chosen to represent a neural network based approach, as that is a growing interest in ML. Additionally, neural networks are already in use for medical classification.

Both results were rounded to the third decimal place, and evaluated using the MulticlassClassificationEvaluator. While there are only two classes, malignant and benign, I found it was easier to implement this evaluator.

Random Forest Results

Precision: 0.960
Recall: 0.980
F1 Score: 0.965
Accuracy: 0.965

Multilayer Perceptron Results

Precision: 0.960
Recall: 0.980
F1 Score: 0.965
Accuracy: 0.965

Both classifiers had identical performance across all metrics. However, this is after I increased the number of iterations for the Multilayer Perceptron. Previously, it performed dramatically lower in all metrics besides recall.

My future improvements would be further optimizing hyperparameters, where I would use cross-validation to ensure the test data is only used to evaluate the final model.